01 / 11B · STEP 1 / THE ORCHESTRATOR MINDSET

Reconnect. Recruit. Make a plan.

Back aboard the Ashwake after the 11A systems seminar, Captain Vale asks for a field test the crew can run. Bring your notes and experiments. Choose your crew, then make one small contract moment playable.

> **The orchestrator mindset**
>
> Bring people, attention, knowledge and tools into an agreed sequence of work. Show what you can contribute, ask for what you need, make a small plan, and follow through. Every member still builds and explains their own scene.

1. **Meet people with something to show.** Open your 11A notes, board or engine experiment. Talk to classmates about a discovery, a difficulty and a story idea. Reconnect with the old team, invite people to a new team, request to join a team, or choose to go it alone. You may call the old team back together. Agree on membership and credit earlier contributions when you reuse them.
2. **Choose one moment worth making playable.** Continue the old story, start a new one, or improve something you already made. State one character, one goal, one player action and one visible consequence. Keep the first scene small enough to finish and test.
3. **Use the whiteboard before the editor.** In a short huddle (about 10 minutes), sketch the scene and the code connection below. Name each contribution, one question to investigate, the tools/resources you will use and when you will compare results. Solo learners make the same plan for themselves.

Character + goal Who is here? What do they want?	**Player action** What can the player do?
Condition When is the action allowed?	**Consequence** What changes on screen?

Example: a Void Runner needs to locate a salvage site in the Training Ring. Move to its damaged beacon; press E within range; the beacon turns cyan to show the signal is restored. Adapt an old contract, propose a new one or improve your own story moment.

The kit includes pen, eraser, shapes, color picker, notes, PNG export and personal print/download controls. It is a local tool; it does not synchronize several people's browsers. A physical board, paper or another shared board is also valid. Each person saves readable screenshots/photos with names and attribution in their personal GitHub.

My team decision and whiteboard plan

My name / personal GitHub:
Old team, new team, request to join, or Working solo:
Agreed members and contributions (an unanswered invitation is still pending):
New / old / improved story; one playable moment:
Character → goal → action → condition → consequence:
What I bring from 11A / what I need help with:
Board file/link / capture method / compare-results time:

▼ **A short invitation you can adapt**

"I tested ___ at the seminar. I want to make ___ from our story playable. I can contribute ___. I need help with ___. Would you like to work together / may I join your team? Let's agree on one small scene and what each of us will show."

These are classroom conversations you conduct yourself. A request is an invitation to agree on work. Working solo is a valid route. Keep your own files and acknowledge earlier collaborators.

Give your story a playable moment.

Build one character or stand-in, one small setting, one interaction and one visible consequence. Keep your first version easy to test.

> **00 / Pre-start · before changing the project**
>
> Open your own project and personal GitHub. Test edit → run → save → commit → push → inspect the remote files. If teamed, test team GitHub and member links too. Preserve your 11A checkpoint. Keep source and required project/dependency settings; use your engine's .gitignore for generated caches, not for authored work.

Unity is the worked example. Use your own engine if you prefer, and map the same concepts to its supported structure. A cube, capsule or sprite can be your character. Extra art and animation can follow after the interaction works.

▼ **Unity · move the runner and activate the beacon**

1. **Save your own starting point.** Test your GitHub connection, commit 11A, then save a new scene named `VoidRunners11B`. Keep the working 11A scene. Use your existing 3D project; a new project is also allowed. Do not leave 11A's automatic movement/rotation scripts on this new player.

2. **Prepare input.** Open Window → Package Manager (some versions: Window → Package Management → Package Manager). Find **Input System** in Unity Registry; install the version recommended for your Editor, or note the version already installed. Enable the new input backend when prompted and accept the Editor restart. If needed, find **Active Input Handling** in Player Settings and choose **Input System Package (New)** or **Both**. Save first. Keep the default dynamic input update setting for this example.

3. **Set the stage.** Create a Plane at (0, 0, 0). Create a Capsule named `Runner` at (0, 1, 0). Remove its Capsule Collider and add a **Character Controller** with default dimensions; do not add a Rigidbody. Create a Cube named `Beacon` at (0, 1, 4). Give Beacon a gray material: URP/Lit in URP, or Standard in Built-in. Keep a Directional Light. Set Main Camera to position (0, 10, −10), rotation (40, 0, 0). The simple shapes are enough for this first scene.

4. **Connect two responsibilities.** Create `Assets/Scripts/VoidRunners11B`. Add the two supplied files below, keeping file and class names identical. Wait for compilation. Attach `PlayerMover` to Runner and `StoryBeacon` to Beacon. Drag Beacon from the Hierarchy into Runner's **Player Mover** → **Beacon** field. Start with Speed 3, Interaction Range 2 and Active Color cyan. Save the scene.

5. **Play the story moment.** Press Play and click the Game view so it receives keyboard input. Use WASD or arrow keys to move; approach Beacon and press E. It turns cyan and logs “communication restored.” Press E far away, then nearby; explain the condition. Stop and restart Play to reset the beacon. Use this as a functional pattern for your story: a salvage beacon, recovered recorder, repaired machine or another object from your contract.

▼ PlayerMover.cs · read, copy or download

Download PlayerMover.cs

```
using UnityEngine;
using UnityEngine.InputSystem; // Types supplied by the Input System package.

[RequireComponent(typeof(CharacterController))]
public class PlayerMover : MonoBehaviour
{
    [SerializeField, Min(0f)] private float speed = 3f;
    [SerializeField, Min(0f)] private float interactionRange = 2f;
    [SerializeField] private StoryBeacon beacon;
    private CharacterController controller;

    public float Speed
    {
        get => speed;
        set => speed = Mathf.Max(0f, value);
    }

    private void Awake()
    {
        controller = GetComponent<CharacterController>();
    }

    private void Update()
    {
        Keyboard keyboard = Keyboard.current;
        Vector3 direction = Vector3.zero;
        if (keyboard != null)
        {
            if (keyboard.wKey.isPressed || keyboard.upArrowKey.isPressed) direction.z += 1f;
            if (keyboard.sKey.isPressed || keyboard.downArrowKey.isPressed) direction.z -= 1f;
            if (keyboard.dKey.isPressed || keyboard.rightArrowKey.isPressed) direction.x += 1f;
            if (keyboard.aKey.isPressed || keyboard.leftArrowKey.isPressed) direction.x -= 1f;
        }

        // SimpleMove takes speed in metres/second and applies gravity and frame timing.
        // Do not multiply this argument by Time.deltaTime.
        controller.SimpleMove(direction.normalized * Speed);

        if (keyboard != null && keyboard.eKey.wasPressedThisFrame && beacon != null)
        {
            float distance = Vector3.Distance(transform.position, beacon.transform.position);
            if (distance <= interactionRange) beacon.Activate();
        }
    }
}
```

▼ StoryBeacon.cs · read, copy or download

Download StoryBeacon.cs

```
using UnityEngine;

[RequireComponent(typeof(MeshRenderer))]
public class StoryBeacon : MonoBehaviour
{
    [SerializeField] private Color activeColor = Color.cyan;
    private Renderer beaconRenderer;
    private MaterialPropertyBlock appearance;

    public bool IsActivated { get; private set; }

    private void Awake()
    {
        beaconRenderer = GetComponent<Renderer>();
        appearance = new MaterialPropertyBlock();
    }

    public void Activate()
    {
        if (IsActivated) return;
        IsActivated = true;

        // Common color properties for URP/Lit and Built-in Standard materials.
        beaconRenderer.GetPropertyBlock(appearance);
        appearance.SetColor("_BaseColor", activeColor);
        appearance.SetColor("_Color", activeColor);
        beaconRenderer.SetPropertyBlock(appearance);
        Debug.Log("Beacon activated: communication restored.", this);
    }
}
```

> **Read the execution path**
>
> Unity calls Awake to cache references, then calls PlayerMover.Update each frame. Keyboard input chooses movement. E plus the distance condition calls this beacon instance's Activate method. IsActivated changes once, and the renderer uses the selected color. The beacon does not need an Update method. Rendering happens through engine components; a C# class alone is not a visible character.

▼ What data, properties and methods are we using?

`speed`, `interactionRange` and `beacon` are serialized fields. `Speed` is a C# property; this Update reads its getter. The Inspector writes the serialized field directly, so it does not run the Speed setter. `IsActivated` is a public read/private write property storing the interaction state. `Update()` and `Activate()` are methods. Dragging Beacon into the Inspector stores a reference to a particular component instance.

`SimpleMove` accepts metres per second and handles gravity/frame timing. Its argument does not need another `Time.deltaTime`. The beacon uses a MaterialPropertyBlock to change this renderer's color without editing the shared material asset. This example supports URP/Lit and Built-in Standard; use your pipeline's matching material.

▼ If something does not work

- **InputSystem or Keyboard is unknown:** confirm package installation and compilation. A using line alone cannot install the package. If your project has custom assembly definitions, also reference the package assembly where needed.
- **Keys do nothing:** focus Game view, check Active Input Handling, confirm the script is enabled and clear Console compile errors. This starter uses a keyboard; another input device can use the same movement/interaction pattern with its supported API.
- **Player falls or collides strangely:** check Plane's collider, remove the extra Capsule Collider/Rigidbody, and restore the starting transforms. Keep movement within the plane.
- **E has no effect:** assign the Beacon reference, move inside the range, and confirm StoryBeacon is on Beacon. Repeated activation is intentionally ignored.
- **Message appears but color does not change:** use the specified Standard or URP/Lit material on Beacon's Mesh Renderer; fix material/shader errors.
- **Package installation is blocked:** keep a record of the error. Use your engine's existing input capability to build the same interaction; inspect the supplied package/using example and explain it. No paid package is required.

▼ Another engine / language · same finish line

1. Continue your own engine project and save a new scene/level. Use a character stand-in plus a target and a camera.
2. Use the engine's supported movement/input setup. In Godot 3D, start with CharacterBody3D, a CollisionShape3D, a visible mesh, a static floor/collider and Camera3D. Define move directions and interact in Project Settings → Input Map; use your engine's movement callback and movement API.
3. Give the target an inactive/active state and an activation method/function. On interaction input, check the distance or overlap, call that target's activation behavior, and show a color, label or other visible change. Add a one-time guard.
4. Record your actual names for object/node/actor, script or behavior unit, editable data, callback, reference and reusable resource. Godot C# commonly uses `using Godot;`; GDScript and other languages use their own mechanisms. Do not copy Unity API names into a different engine.

5. Identify a built-in or added input library/module/resource and its version/source. Explain how it becomes available and how your code refers to it. You do not need to install a Unity package or switch engines.

Use your engine's official getting-started documentation for its input/controller setup. Show the same character → player action → condition → visible consequence. Explain any different structure.

My scene and code connection

Engine / version / language:
Scene / character / target:
Which field stores my value? Which property or method uses it?
Input → callback → condition → method → visible result:
Code I authored / adapted; source attribution:
Evidence file(s) showing my own working scene:

Know what you borrowed.

Developers reuse other people's work. Explain what a resource supplies, how your code refers to it, and what you are responsible for.

Package / library
Provides the input capability

Namespace
UnityEngine.InputSystem

Type / class
Keyboard

Reference + member
Keyboard.current → wKey.isPressed

Different jobs in the same path

Part	What it does	Our example
Package	Delivers code, tools, assets or a combination. A package containing art may introduce no C# namespace.	Input System: `com.unity.inputsystem`
Assembly / project reference	Compiled code must be available to the script. Unity normally handles default project references; custom assemblies may need explicit references.	The Input System code is available to PlayerMover.
Namespace + using	Organizes type names; a using directive lets this file use shorter type names.	`using UnityEngine.InputSystem;`
Class / type	Describes a kind of object and the members it provides.	`Keyboard`, `PlayerMover`, `StoryBeacon`
Instance / component	A particular runtime object holds its own state. Attaching the script supplies a behavior component.	Runner's PlayerMover holds a reference to Beacon's StoryBeacon.
Method / property	Performs an action or supplies/changes a value.	`Activate()`, `Speed`, `IsActivated`

```
// Same type name, written two ways:
UnityEngine.InputSystem.Keyboard keyboard =
    UnityEngine.InputSystem.Keyboard.current;

// With using UnityEngine.InputSystem; at the top of the file:
Keyboard keyboard = Keyboard.current;
```

A using directive changes how we write names in this file. It does not download a package, create a Keyboard, attach a component, or start Update. `Keyboard.current` supplies a reference to the current keyboard device. An engine's package system, project references and runtime have separate jobs. The `using` keyword also has a resource-disposal form in C#; today we are studying namespace directives.

▼ **A small investigation you can explain**

1. Find `com.unity.inputsystem` in Package Manager and `Packages/manifest.json`. Record the installed version and official source. Observe the dependency; do not delete it.
2. Read the two using lines in PlayerMover. Identify which namespace supplies `CharacterController` and which supplies `Keyboard`.
3. Explain why the fully qualified Keyboard example names the same type. Removing a using can require full type names; removing a package can remove the code itself.
4. Point to code you wrote or changed, code supplied by this assignment and code supplied by the engine/package. Keep a short attribution note.

> **Use one resource deliberately**
>
> The Input System package is enough for this example. You may also use a character or environment asset, but keep its source, author/license and any changes in your README. A supplied character may include scripts, settings and dependencies; inspect them before adding your own controller. Keep one movement controller active on the character.

My package / using explanation

Capability / package or built-in resource / installed version:
Official source and asset attribution (if any):
What it provides and where my project records the dependency:
Namespace or import mechanism / type I use:
What the using directive does; what installs the code:
What I reused / what I wrote or changed:
If blocked: exact error, equivalent capability and my explanation:

▼ **Official references**

- Microsoft: C# using directives
- Unity: packages and package management
- Input System: installation (use your installed version’s documentation)
- Input System: keyboard
- CharacterController.SimpleMove
- Renderer.SetPropertyBlock
- Godot: C# basics

Test it. Improve it. Bring it back.

Return with something another person can run and understand. Show what changed, what worked and what still needs attention.

1. **Run four checks.** Move in at least two directions. Try the interaction while outside its range, then inside. Try activation a second time. Record the expected result and what happened; repeated activation should keep the beacon active.

2. **Improve one thing.** Change Speed, Interaction Range, Active Color or one equally small part of your story interaction. Predict before running; record the result or exact error and explain it. Extra movement, jumps, physics, characters or effects are optional after the base works.

3. **Return to your collaborators.** Compare the whiteboard promise with your actual scene. Demonstrate one working behavior, show one problem and explain your next decision. Team members can share discoveries and improve the story; each learner keeps a separate working build and personal evidence.

4. **Push and check your handoff.** Put your scene/project source, scripts, required dependency settings, personal notes, readable whiteboard captures, test evidence and source/asset credits in your personal GitHub. Use a README with run instructions. Test the pushed files/images/links and instructor access. If teamed, put the tested team GitHub link in your personal README and link member repositories from the team index. Solo learners write “Working solo.” Submit **one personal GitHub repository URL** in your own Canvas 11B record.

My test, improvement and team return

Movement result:
Outside range / inside range / repeated activation results:
One changed value: prediction → observed result/error → explanation → next step:
What I promised → what I showed:
What I learned from another person (with credit), or solo review:
Run instructions / evidence and board file names:
Personal GitHub / team GitHub or Working solo:
Before-work Git test / final push, remote link and instructor-access check:

Each member can download or print their own notes and keep their own whiteboard PNG or photo. These are personal copies; downloads and checked boxes do not submit work or verify completion. Review the README draft and add your real run instructions and evidence before pushing.

▼ **Personal GitHub · files to keep**

```
README.md                  # story moment, run steps, tested team link or Working solo
project/                   # editable engine project, scene and scripts
notes/11b-notes.md          # personal decisions, package explanation, tests
evidence/whiteboard.png     # or readable board photographs
evidence/scene-running.png  # plus setup/details or a short demonstration
credits.md                 # reused code, packages, assets and contributions
```

Unity: keep authored `Assets` including .meta files, `Packages/manifest.json`, the package lock file and required `ProjectSettings`. Exclude generated Library/Temp folders with an appropriate Unity .gitignore. Other engines: keep the equivalent source, scene and dependency settings. Do not upload passwords, account tokens or unrelated private files.

▼ **Four requirements · complete wording**

R1 · Reconnect and map one playable moment · 3 points

Choose the old team, a newly agreed team, a request to join others, or Working solo. Record your contribution and who agreed to what; invitations need an answer before someone is listed as a member. Use a new story, an old story or an improved version. Save a readable whiteboard sketch linking character → goal → player action → condition → visible consequence, with named contributions. Each learner keeps their own copy and a short explanation in their personal GitHub.

R2 · Build and explain your own playable scene · 6 points

Individually build one small scene in your chosen engine: one visible character or stand-in, player-controlled movement, one story interaction, and one visible consequence. Reuse your 11A learning. Show the connected scripts/behavior units, relevant data and method, and a working result. A shared team scene cannot replace your individual build.

R3 · Trace a borrowed capability · 3 points

For Unity, use the Input System package in the worked example, or an already-available engine capability if installation is blocked. Record its name/version/source, what you reused and what you authored. Explain package/library, namespace, type/class, component instance and method using your actual project. Explain what a C# using directive changes and why it does not install code or create an instance. Another engine/language may use its own import/module mechanism; identify the equivalent and its differences.

R4 · Test, improve and hand off your work · 3 points

Test movement, interaction outside and inside its range, and repeated activation. Change one meaningful value and record prediction → observed result/error → explanation → next step. Keep source, scene, package/dependency settings, notes, board captures, test evidence and attribution in your personal GitHub. Test the local/push/remote workflow before work and verify the final pushed files, images, links and instructor access. If teamed, put the tested team GitHub link in your personal README and link personal repositories from the team index. Submit one PERSONAL GitHub URL in your own Canvas record. Solo learners need no team GitHub.

> **Canvas is the submission record**
>
> Submit the one personal GitHub URL through Action 11B. 15 points · Due Wednesday, November 4, 2026 at 6:30 PM Pacific. Submit through your own Canvas record and verify the submission confirmation. Canvas holds the official deadline and grade.

Return to Canvas Action 11B ↗